End-to-End Time Series Forecasting System with REST API

INTRODUCTION

The **End-to-End Time Series Forecasting System with REST API** is a machine learning forecasting application developed to forecast future sales trends for different US states using historical time-series data. The project combines machine learning, feature engineering, REST API development, and interactive dashboard visualization into a complete forecasting pipeline.

The system predicts the next 8 weeks of sales using advanced forecasting techniques and exposes predictions through a Flask REST API integrated with a Streamlit frontend dashboard.

US Dataset & Modeling Approach

- **US Dataset**: The forecasting pipeline is trained and evaluated on a comprehensive US sales dataset (see `data/final_feature_engineered_data.xlsx`) covering 43 US states and historical records from 2019 through 2023. The dataset contains raw sales history plus engineered features used as model inputs.
- **Models Implemented**: Four forecasting approaches were developed and evaluated in the project:
  - **SARIMA** — seasonal ARIMA used as a classical statistical baseline for univariate series.
  - **Prophet** — additive trend/seasonality model from Facebook/Meta for robust seasonality handling.
  - **XGBoost** — gradient-boosted tree regressor using the engineered feature set (production-ready, fast inference).
  - **LSTM** — recurrent neural network capturing longer temporal dependencies in sequences.
- **Automatic Best-Model Selection**: For each state the pipeline evaluates models on validation metrics (e.g., MAE, RMSE, MAPE) and automatically selects the best-performing model — the one with the lowest chosen metric. The API response includes `best_model` so the frontend and users know which model produced the returned forecasts.

OBJECTIVE

- The main objective of this project is to build a production-ready forecasting system that:
- Trains multiple forecasting algorithms

- Compares and selects the best model
- Generates accurate future sales forecasts
- Exposes predictions through a REST API
- Provides an interactive frontend dashboard
- Simulates a real-world backend forecasting service

PROBLEM STATEMENT

- Forecast the next 8 weeks of sales for each US state using historical time-series data. The forecasting system must:
- Handle missing values and dates
- Capture seasonality and trend
- Automatically select the best-performing model
- Provide predictions through an API
- Support frontend visualization and analytics

OVERVIEW

This project delivers an end-to-end machine learning forecasting pipeline that predicts sales trends across US states. The system combines:

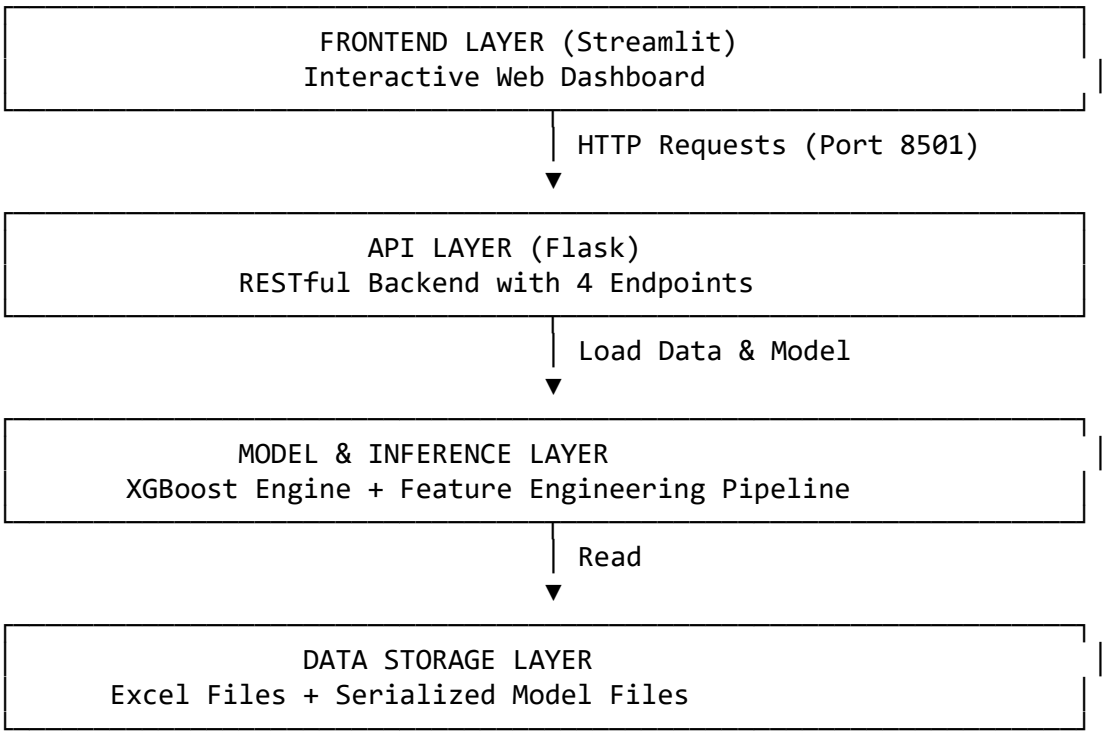
- **Historical Time-Series Data**: Feature-engineered datasets spanning multiple years
- **XGBoost ML Model**: Production-ready gradient boosting model for accurate predictions
- **Flask REST API**: Scalable backend serving forecasting predictions
- **Streamlit Dashboard**: Interactive UI for exploring state-specific forecasts

The solution enables data-driven decision-making by providing 8-week sales forecasts for any US state with dynamic feature engineering and real-time model inference.

ARCHITECTURE

System Architecture Overview

The system follows a **4-layer architecture** designed for scalability, maintainability, and scalable deployment:



DATA FLOW

1. **User Interaction**: Select state and forecast weeks in Streamlit
2. **Frontend Request**: Send `GET /predict/<state>` to Flask API
3. **Data Retrieval**: Load state-specific features from Excel
4. **Feature Engineering**: Extract and prepare 9 input features
5. **Model Inference**: XGBoost generates predictions for each week
6. **Dynamic Updates**: Lag values and rolling stats updated sequentially
7. **API Response**: Return JSON with 8-week forecasts and metrics
8. **Visualization**: Render table, metrics, and interactive chart

FEATURES

Frontend (Streamlit)

- **Dynamic State Selection**: Load all 43 available US states from API
- **Flexible Forecasting**: Adjust forecast horizon (4-8 weeks) via slider
- **Interactive Visualizations**: Line chart with Plotly showing forecast trends
- **Performance Metrics**: Display best model, average forecast, peak forecast
- **Responsive Design**: Interactive and responsive Streamlit dashboard
- **Professional Captions**: Informative descriptions for API and developer

REST API (Flask)

- **State Management**: `/states` endpoint returns all available states

- **Health Check**: `/` and `/help` routes for API status and documentation
- **Smart Predictions**: `/predict/<state>` generates 8-week forecasts
- **Case-Insensitive State Matching**: Robust state lookup
- **Date-Anchored Forecasts**: Forecasts start from state's latest data date
- **Dynamic Feature Updates**: Lag values and rolling statistics updated per week

Machine Learning Model

- **XGBoost Gradient Boosting**: Machine learning forecasting model
- **Feature Set**: 9 input features including lags, rolling stats, and temporal info
- **Date-Aware Predictions**: Sequential forecasts with dynamic feature evolution
- **Scalable Inference**: Fast predictions for real-time use cases

Data Engineering

- **Feature Engineering**: Lag features (1, 7, 30 days), rolling mean/std
- **State Encoding**: Numerical encoding for categorical state information
- **Temporal Features**: Day-of-week, month, holiday flags
- **Data Quality**: Cleaned historical data spanning 2019-2023

TECHNOLOGIES USED

Backend

Technology	Purpose
Python	Core programming language
Flask	REST API framework
XGBoost	Machine learning model
Pandas	Data manipulation
NumPy	Numerical computing
Scikit-learn	ML utilities
joblib	Model serialization

Frontend

Technology	Purpose
Streamlit	Interactive web UI
Plotly	Interactive visualizations
Pandas	Data handling
Requests	HTTP client

Data & Storage

Format	Purpose
XLSX (Excel)	Feature-engineered data storage
PKL (Pickle)	Serialized model storage

SYSTEM OUTPUTS

The developed forecasting system successfully generates dynamic state-wise sales forecasts using historical time-series data and machine learning models. The application provides predictions through a Flask REST API and visualizes the forecasting results using an interactive Streamlit dashboard.

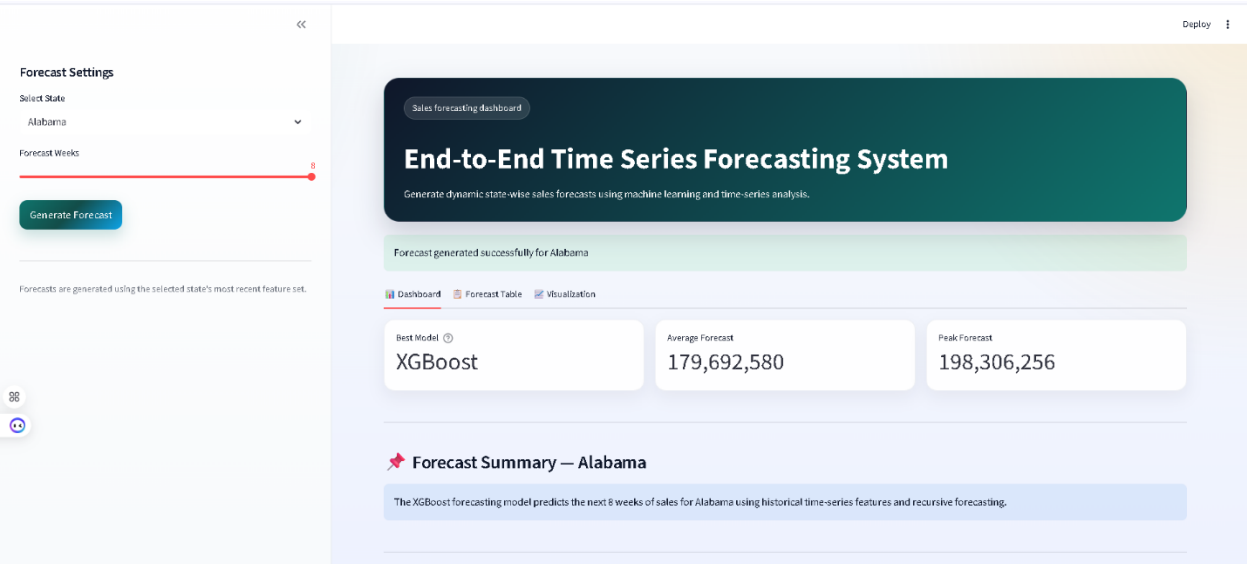
Dashboard Output

The Streamlit dashboard allows users to:

- Select a US state dynamically
- Choose forecast horizon weeks
- Generate forecasts interactively
- View forecast metrics and visualizations

Dashboard Features

- Dynamic state selection
- Forecast generation button
- Interactive Plotly graph
- Forecast metrics cards
- Forecast table visualization

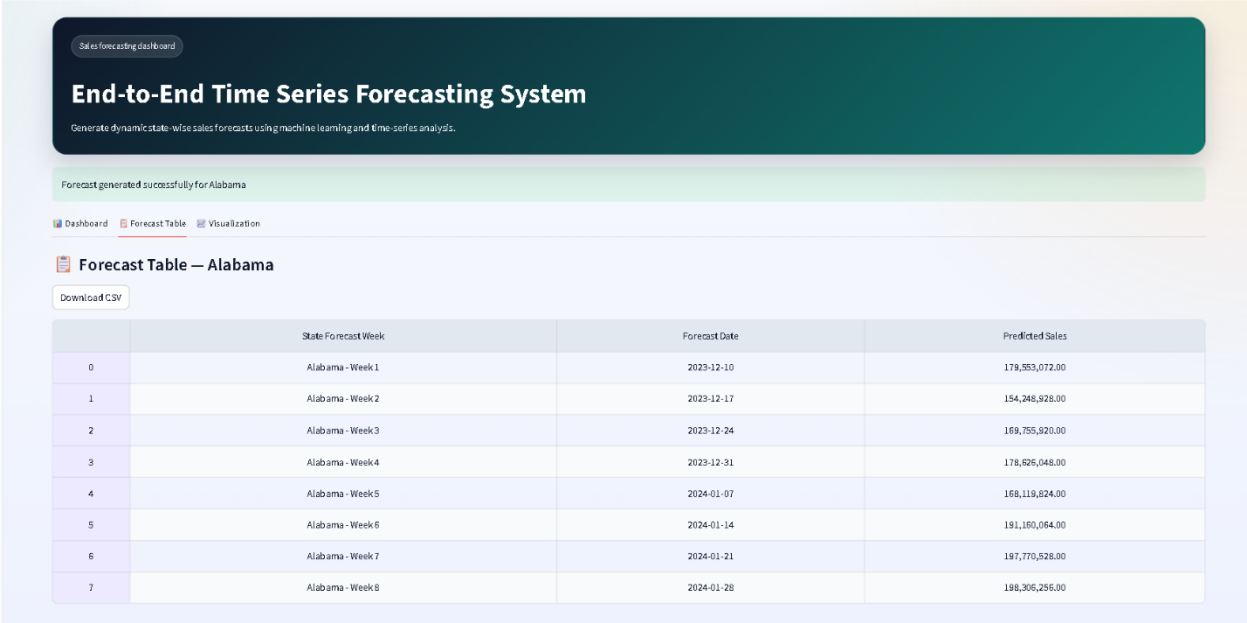


Forecast Table Output

The dashboard displays forecasted sales values for upcoming weeks in tabular format.

Table Information

Column	Description
State Forecast Week	Forecast week label
Forecast Date	Future prediction date
Predicted Sales	Forecasted sales value



	State Forecast Week	Forecast Date	Predicted Sales
0	Alabama - Week 1	2023-12-10	179,553,072.00
1	Alabama - Week 2	2023-12-17	154,248,928.00
2	Alabama - Week 3	2023-12-24	169,755,920.00
3	Alabama - Week 4	2023-12-31	178,626,048.00
4	Alabama - Week 5	2024-01-07	168,119,824.00
5	Alabama - Week 6	2024-01-14	191,160,064.00
6	Alabama - Week 7	2024-01-21	197,770,528.00
7	Alabama - Week 8	2024-01-28	198,306,256.00

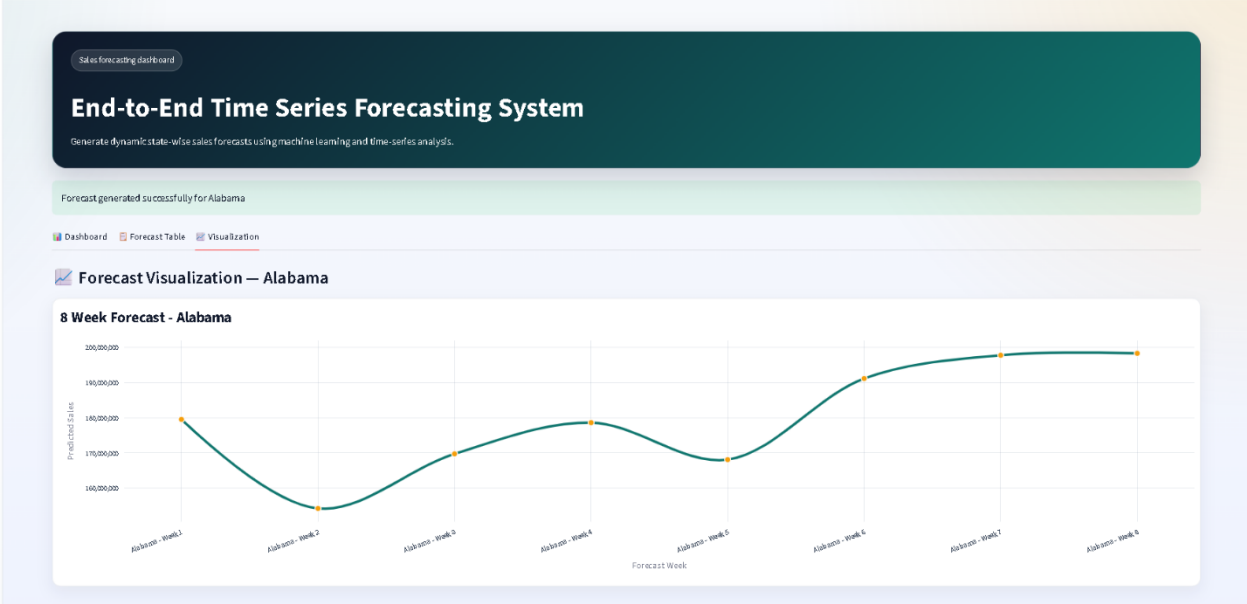
Forecast Visualization Output

The system generates an interactive line chart using Plotly to visualize future sales trends.

Graph Features

- Interactive hover information
- Dynamic line forecasting
- State-wise forecast visualization

- Responsive chart rendering



RESULTS

The forecasting system was successfully developed and tested using historical state-wise sales data. Multiple forecasting models were implemented and evaluated to identify the best-performing approach.

Model Performance Comparison

Model	Performance
SARIMA	Moderate forecasting accuracy
Prophet	Good seasonality handling
XGBoost	Best overall performance
LSTM	Computationally expensive but effective

XGBoost achieved the best balance between:

- forecasting accuracy
- inference speed

- scalability
- dynamic prediction capability

Therefore, XGBoost was selected as the final deployment model for the forecasting system.

CONCLUSION

The project successfully implemented a complete end-to-end forecasting pipeline capable of predicting future state-wise sales using machine learning and time-series techniques.

The system combines:

- feature engineering
- machine learning forecasting
- REST API integration
- interactive frontend visualization

into a production-style forecasting architecture. The project demonstrates practical implementation of machine learning engineering, backend API development, and frontend analytics dashboard integration.